

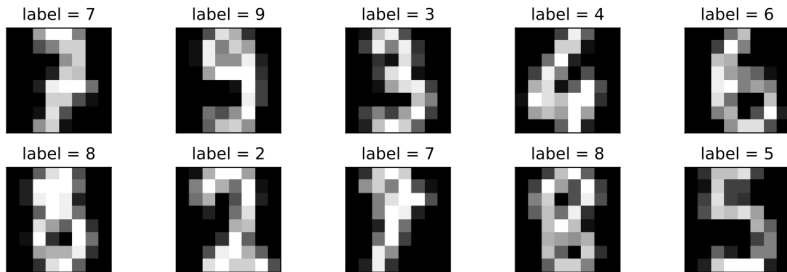
Classification of Handwritten Digits using Stochastic Gradient Descent

Álvaro Galisteo Bermúdez, Daniel Rath

16.02.2026

The Dataset

The dataset contains 1797 grayscale 8×8 images of handwritten digits. Each associated with a label.



It is available in python through the sklearn library

```
from sklearn.datasets import load_digits
```

Multiclass Logistic regression - I

We represent the label y_j of the image a_j as

$$p_j \in \mathbb{R}^{10}, \quad (p_j)_i = \begin{cases} 1 & \text{if } i = y_j \\ 0 & \text{otherwise} \end{cases}$$

The scores are given as

$$\varphi(a_j, x)_i = a_j^T w_i + b_i \quad \text{with } x = (w_i, b_i)_{i=1}^{10} \quad w_i \in \mathbb{R}^{64}, b_i \in \mathbb{R}$$

The prediction is given by

$$\tilde{y}_j = \arg \max_{i \in \{0, \dots, 9\}} \varphi(a_j; x)_i$$

We interpret the scores as probabilities by

$$\sigma(\varphi(a_j; x))_i = \frac{\exp(\varphi(a_j; x)_i)}{\sum_{l=0}^9 \exp(\varphi(a_j; x)_l)}$$

Multiclass Logistic regression - II

The Optimization problem

The optimization problem we want to solve is

$$\underset{\mathbf{x} \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \quad \frac{1}{N} \sum_{j=1}^N L(p_j, \varphi(a_j; \mathbf{x})),$$

where L is the cross-entropy loss defined as

$$L(p, s) = \log \left(\sum_{l=0}^9 \exp(s_l) \right) - \sum_{i=0}^9 p_i s_i$$

Cross entropy loss

$$L(p, s) = \log \left(\sum_{l=0}^9 \exp(s_l) \right) - \sum_{i=0}^9 p_i s_i$$

Using the cross-entropy loss ensures some important properties:

- Convexity of the optimization problem
- Differentiability for gradient descent methods
- Maximization of the likelihood of the observed data during training

Also note that we do not use the $\tilde{y} = \arg \max_{i \in \{0, \dots, 9\}} \varphi(a_j; x)_i$ in the loss function, because it is not differentiable.

Stochastic Gradient Descent

For solving the optimization problem we use Stochastic Gradient Descent (SGD), i.e. we choose a mini-batch $\mathcal{B}_k$ of size B and calculate a stochastic estimator of the gradient as

$$\nabla L(x) = \frac{1}{B} \sum_{j \in \mathcal{B}_k} \nabla L(p_j, \varphi(a_j; x))$$

The update is then given by

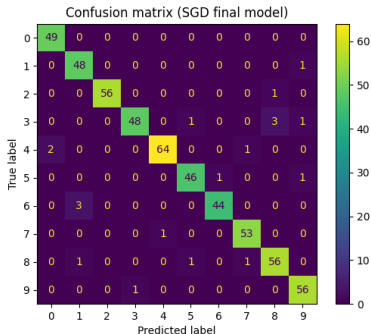
$$x^{k+1} = x^k - \eta_k \nabla L(x^k)$$

This approach has some advantages and disadvantages, e.g.:

- Computationally efficient for larger datasets and small B
- Gradient can be too noisy if B is too small

Results with SGD

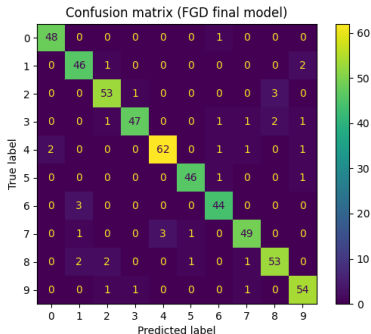
We trained the model for 50 epochs with a mini-batch size of $B = 64$ and a learning rate of $\eta_k = 0.5$ on a random training set of 70% of the data. The remaining 30% was used as a test set.



- Accuracy (train data): 0.973
- Accuracy (test data): 0.962
- Training Time: TODO

Results with FGD

We trained the reference model for 300 iterations with a learning rate of $\eta_k = 0.5$ on a random training set of 70% of the data. The remaining 30% was used as a test set.



- Accuracy (train data): 0.934
- Accuracy (test data): 0.929
- Training Time: TODO

Convergence Comparison

